

Next.js and Django Architecture Diagram

<https://gemini.google.com/app/3f6856ed4cb30096>

User prompt: Turn this into a Mermaid diagram. Client, a Next.js app, talks to an nginx load balancer, which talks to two or more Django and DRF instances, which talk to Postgres. A Celery worker reads from Redis to handle send jobs asynchronously. A view or the worker makes an outbound call to an external AI provider to generate a draft, with a timeout path back to writing the announcement manually if that call fails. Label each connection as synchronous or asynchronous. Keep it accurate to what is actually described in the earlier sessions, not extra pieces I have not built.

Response: graph TD

%% Client & Routing Layer

Client["Client App
(Next.js)"]

Nginx["Load Balancer
(Nginx)"]

%% Application Server Cluster

subgraph DjangoCluster ["Django App Servers"]

Django1["Django & DRF Instance 1"]

Django2["Django & DRF Instance N"]

end

%% Data Stores

Postgres[(Database
Postgres)]

Redis[(Cache & Message Broker
Redis)]

%% Async Worker

Worker["Background Worker
(Celery)"]

%% External AI Service

AIProvider["External AI Provider"]

%% Connections - Client to Infrastructure

Client -- "Synchronous" --> Nginx

Nginx -- "Synchronous" --> Django1

Nginx -- "Synchronous" --> Django2

%% Connections - Django Instance Data Access & Queueing

Django1 -- "Synchronous" --> Postgres

Django2 -- "Synchronous" --> Postgres

Django1 -- "Synchronous" --> Redis

Django2 -- "Synchronous" --> Redis

%% Connections - Celery Worker Processing & Storage

Worker -- "Asynchronous" --> Redis

Worker -- "Synchronous" --> Postgres

%% Connections - Outbound AI Generation & Timeout Path

Django1 -. "Synchronous
(AI Draft Request)" .-> AIProvider

```

Worker -. "Synchronous<br/>(AI Draft Request)" .-> AIProvider
AIProvider -. "Synchronous<br/>(Timeout Path / Manual Fallback)" .-> Django1
AIProvider -. "Synchronous<br/>(Timeout Path / Manual Fallback)" .-> Worker

```

Connection Summary

- **Synchronous Connections:**

- **Next.js Client** → **Nginx**: Standard HTTP(S) request/response cycle.
- **Nginx** → **Django & DRF Instances**: HTTP proxy routing to available web app instances.
- **Django Instances** → **Postgres**: Direct SQL queries for read/write operations and authentication.
- **Django Instances** → **Redis**: Enqueueing task payloads (via `task.delay()`) and standard Redis key lookups/caching.
- **Celery Worker** → **Postgres**: Direct batch row generation (`INSERT ON CONFLICT DO NOTHING`) and database updates.
- **Django View / Celery Worker** ↔ **External AI Provider**: Blocking outbound HTTP request to generate draft text. If the request times out or throws an error, execution immediately falls back synchronously to the manual entry path.

- **Asynchronous Connections:**

- **Celery Worker** → **Redis**: Decoupled background queue polling/reading to retrieve enqueued background tasks.
-